CS211FZ

DATA STRUCTURES & ALGORITHMS II

Dynamic Programming

Lecturer: YINGLIAN DENG · Yinglian.Deng@mu.ie

Introduction

Yinglian Deng – Full-Stack Developer & Researcher, Maynooth University

· 8 years work experience in software development, and work at the intersection of software engineering, AI, and digital education.

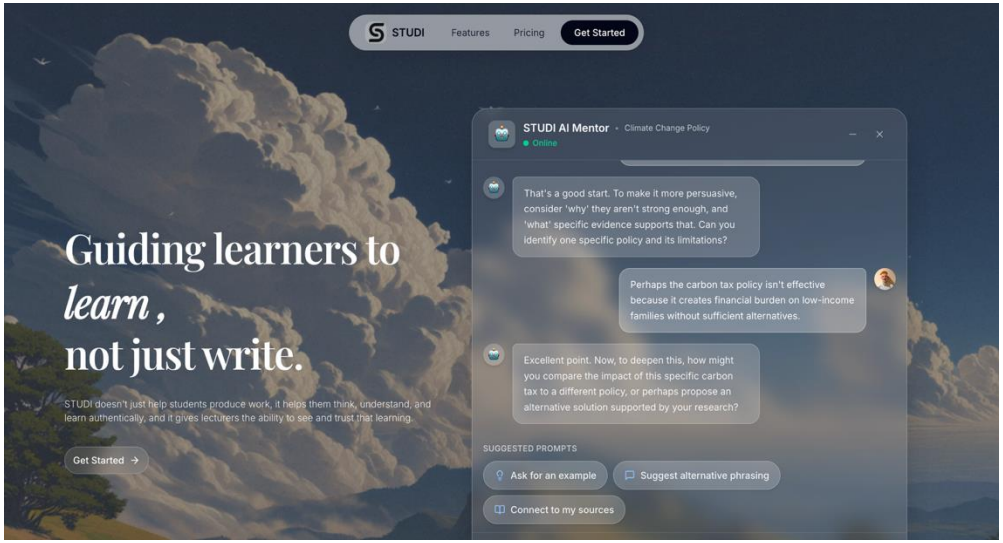
· Research focuses on how generative AI is reshaping higher education, shifting assessment from evaluating final outputs to understanding the learning process.

Research Projects: Metanode, Studi

· Metanode: a platform that uses AI agents to generate personalised learning pathways. It includes workflow and AR.

· Studi: A digital assessment environment where the learning process is visible over time.

· Studi: Uses lecturer-designed pedagogy bots that guide students as AI-supported learning partners, encouraging reflection, judgement, and ethical AI use.



Why Data Structures & Algorithms Matter

Systems like AI agents, learning pathway generation, and engagement tracking rely on efficient algorithms and structured data. These computational foundations help us design scalable and intelligent learning systems.

Dynamic Programming — Definition

Who invented it?

Richard Bellman

Mathematician & Computer Scientist
1920 – 1984

Richard Bellman introduced the idea of dynamic programming in the 1950s.

"I spent the Fall quarter at RAND thinking about planning processes, and I started calling the theory 'dynamic programming'."

— Bellman, 1952

What is it?

An algorithm design paradigm for solving complex optimisation problems.

① **Decompose**

Break a complex problem into a collection of simpler "overlapping" subproblems.

② **Solve Once**

Solve each distinct subproblem only once — never recompute what you already know.

③ **Store**

Record the result of each subproblem in a table (memoisation or tabulation).

④ **Reuse**

Use the stored answers to build the solution to larger subproblems and finally the overall solution.

Dynamic Programming — Application Areas

DP is everywhere — from your phone to your DNA

💻 Computer Science & AI

- Artificial Intelligence & Machine Learning
- Natural Language Processing
- Compilers & Operating Systems
- Computer Graphics & Robotics

🔬 Science & Data

- Bioinformatics — DNA / protein sequence alignment
- Information Theory
- Computational Biology
- Statistical Modelling

📈 Optimisation & Decision Making

- Operations Research
- Multistage Decision Processes
- Control Theory
- Resource Scheduling

🏭 Industry Applications

- Finance — portfolio optimisation, pricing models
- Network routing & shortest path
- Game development & pathfinding
- Supply chain optimisation

🎯 **Practical note: DP appears frequently in technical coding interviews (Google, Meta, Amazon, Microsoft)— mastering it is career-critical.**

Dynamic Programming — Famous Algorithms

DP powers many algorithms you already use every day

Graph Theory

Bellman–Ford–Moore

Computes shortest paths from a single source to all other vertices. Unlike Dijkstra's, handles negative edge weights.

Used in: Network routing, GPS navigation

Bioinformatics

Needleman–Wunsch & Smith–Waterman

Global and local sequence alignment algorithms for comparing DNA, RNA, and protein sequences.

Used in: Genomics, drug discovery

Signal Processing / AI

Viterbi Algorithm

Finds the most likely sequence of hidden states in a Hidden Markov Model using DP to avoid exponential search.

Used in: Speech recognition, NLP decoding

Databases

System R (Join Order)

Selects the optimal join order for multi-table SQL queries by storing the cost of all sub-join combinations.

Used in: Every SQL database engine

Computer Graphics

De Boor's Algorithm

Numerically stable evaluation of B-spline curves using a DP recurrence — avoids catastrophic cancellation.

Used in: CAD software, font rendering

Software Engineering

Unix diff

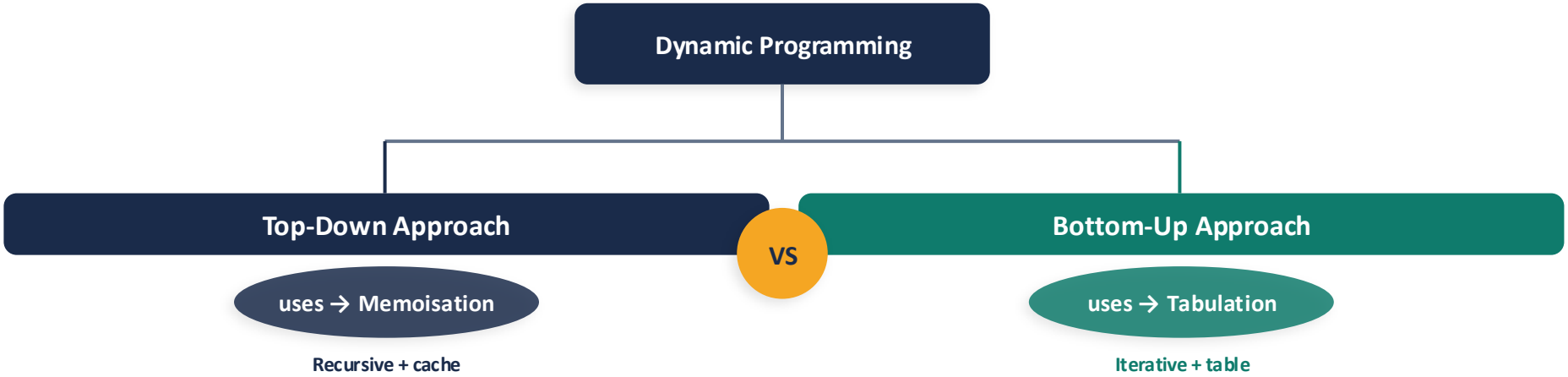
Computes the Longest Common Subsequence (LCS) of two files to produce a minimal edit script.

Used in: Git, code review tools

Dynamic Programming — Two Techniques to Solve Problems

Both store and reuse subproblem results — but differ in direction

DP is divided into two main approaches. Both eliminate redundant computation, but they work in opposite directions.



How it works:

Start from the original problem and recursively break it down. Before computing a subproblem, check if the answer is already cached. If yes — return it. If no — compute, store, return.

Example

```
Fibonacci: O(n) time, O(n) space
```

How it works:

Start from the smallest base cases and iteratively build up to the final answer. Fill a table in a predetermined order so that each entry only depends on already-computed entries.

Example

```
Fibonacci: O(n) time, O(1) with rolling array
```

Example — Fibonacci Numbers

The classic introduction to DP: F(i) = F(i-1) + F(i-2)

Mathematical Definition

`F(i) = 0`	*if i = 0*
`F(i) = 1`	*if i = 1*
`F(i) = F(i-1) + F(i-2)`	*if i > 1*

Sequence:

0	1	1	2	3	5	8	13	21	34	...
F(0)	F(1)	F(2)	F(3)	F(4)	F(5)	F(6)	F(7)	F(8)	F(9)	

Fibonacci in Nature

- **3** petals — Lily, Iris
- **5** petals — Buttercup, Columbine
- **8** petals — Delphinium, Bloodroot
- **13** petals — Marigold, Cineraria
- **21** petals — Aster, Chicory
- **34–89** petals — Sunflower, Daisy spirals

Why Fibonacci is the Perfect DP Example

✅ Optimal Substructure

F(n) is defined entirely in terms of F(n-1) and F(n-2). The solution to F(n) is built directly from solutions to smaller subproblems.

✅ Overlapping Subproblems

Computing F(5) naively calls F(3) and F(2) multiple times. The same subproblems repeat — this is the key DP indicator.

✅ Simple State Definition

dp[i] = "the i-th Fibonacci number". One-dimensional. Crystal clear. A perfect first DP state to learn.

Complexity comparison:

- **Naive recursion:** `Time O(2ⁿ) | Space O(n) stack`
- **DP tabulation:** `Time O(n) | Space O(1) rolling array`

Fibonacci — Naive Recursion Has a Problem

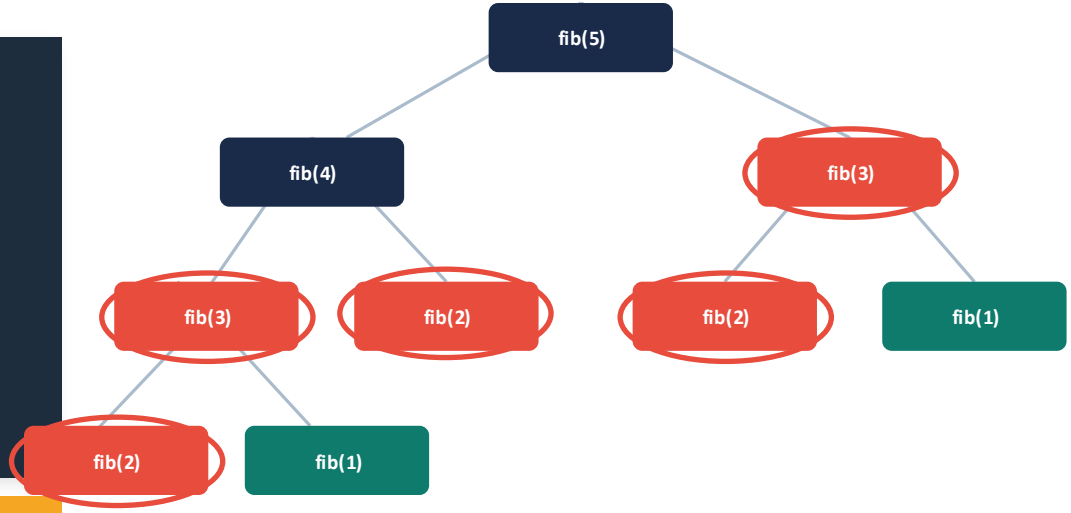
fib(n) = fib(n-1) + fib(n-2) | Can you spot the inefficiency?

Recursive code:

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}

// Looks simple and correct...
// But fib(40) makes ~2.2 billion calls!
// Time complexity: O(2^n)  — exponential 😱
```

Recursion tree for fib(5):



Question: "What do you notice about this call tree?"

Expected answer:

fib(3) and fib(2) are computed MULTIPLE TIMES.
This is "Overlapping Subproblems" — the key DP indicator!

= Computed more than once! → Wasted work

```
fib(3) computed: 2×   |   fib(2) computed: 3×   |   fib(1) computed: 3×
```

Two Implementation Approaches

Memoization (Top-Down) vs Tabulation (Bottom-Up)

Top-Down (Memoization)

```
int[] memo;

int fib(int n) {
    if (n <= 1) return n;
    // Check memo table first
    if (memo[n] != -1) return memo[n];
    // Store result
    memo[n] = fib(n-1) + fib(n-2);
    return memo[n];
}
// Time O(n)  Space O(n)
```

Key Traits

- Recursive — natural and intuitive
- Lazy evaluation (compute on demand)
- Risk of stack overflow for large n

Bottom-Up (Tabulation)

```
int fib(int n) {
    if (n <= 1) return n;
    int[] dp = new int[n+1];
    dp[0] = 0;  dp[1] = 1;
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i-1] + dp[i-2];
    }
    return dp[n];
}
// Time O(n)  Space O(n)
// Space can be optimised to O(1)
```

Key Traits

- Iterative — no stack overflow risk
- Fills table in order, stable performance
- Easy to apply rolling-array space opt.

Two Core Concepts of DP

Necessary conditions for a problem to be solvable with DP

① Optimal Substructure

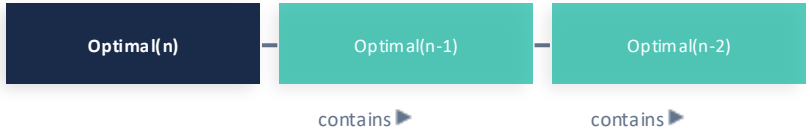
Optimal Substructure

Definition

The optimal solution to the original problem can be built from optimal solutions of its subproblems.

Example

0/1 Knapsack: the best selection of i items includes the best selection of i-1 items.



② Overlapping Subproblems

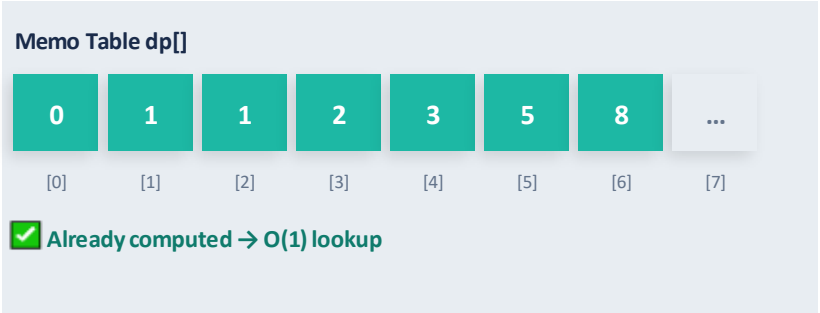
Overlapping Subproblems

Definition

The same subproblems are solved repeatedly during recursion.

Example

Fibonacci: fib(3) is called multiple times when computing fib(5).



Classic Problem

Longest Common Subsequence Problem

Longest Common Subsequence - Problem

Longest Common Subsequence — LeetCode 1143

Problem:

- Let's consider an application of dynamic programming, where the order of solving the subproblems is not so clear, unlike Fibonacci numbers.
- Solving these problems will be difficult without recursive thinking and a bottom-up dynamic programming approach.
- Break down the problem into smaller subproblems, solve them in a specific order.

Longest Common Subsequence - Problem

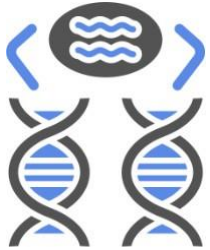
String Similarity Applications

- A fundamental string-processing problem that arises in computational biology and other domains.
- Given two strings x and y, we wish to determine how similar they are.

Some examples:

- Comparing two DNA sequences for homology
- Two English words for spelling
- Two Java files for repeated code

1 DNA Sequences

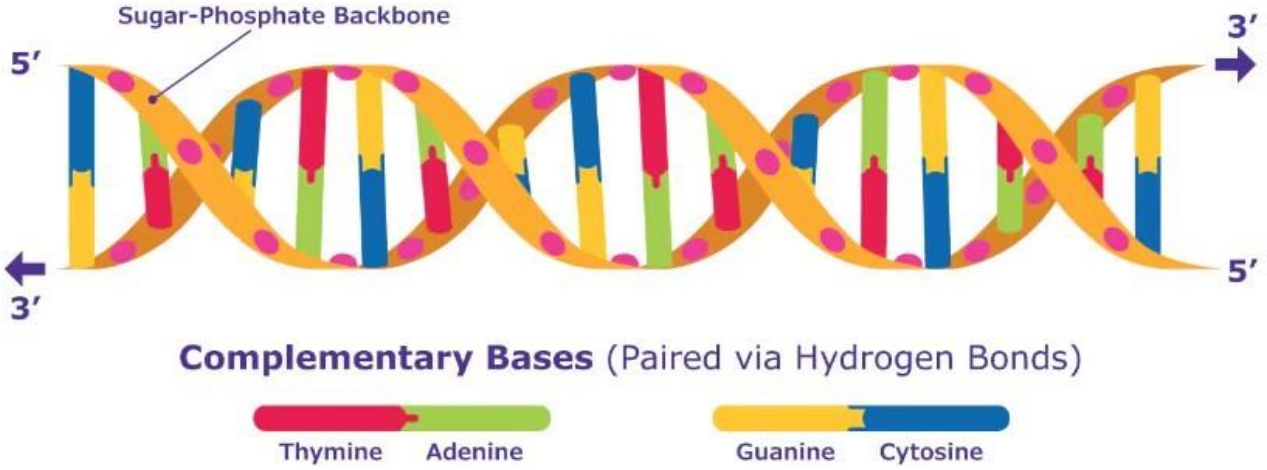


2 Spell Checking

3 Code Diff

Longest Common Subsequence - Problem

- Biological applications often need to compare the DNA of two (or more) different organisms.
- A strand of DNA consists of a string of molecules called bases.
- Possible bases are:
- Adenine {A}
- Cytosine {C}
- Guanine {G}
- Thymine {T}



DNA Sequence Comparison

- We can express a strand of DNA as a string over the 4-element set {A, C, G, T}.
- For example, the DNA of one organism may be:

S_1 = ACCGGTCGAGTGCGCGGAAGCCGGCCGAA

- and the DNA of another organism may be:

S_2 = GTCGTTCGGAATGCCGTTGCTCTGTAAA

- *Compare two strands of DNA to determine how "similar" or how "closely" related the two organisms are.*

What is a Subsequence?

- A subsequence of a string S is a set of characters that appear in left-to-right order, but not necessarily consecutively.

Example string:

A C T T G C G

✅ **Valid subsequences:**

ACT | ATTC | T | ACTTGC

❌ **Not a subsequence:**

TTA (T comes before T, but A appears before the second T in ACTTGCG)

Longest Common Subsequence - Introduction

Substring VS **Subsequence**

No gaps

TON

Gaps allowed

TOE

Example

STONE

TOWER

How it works:

toe is a common subsequence of length three.
toe is the longest common subsequence for these two words.

Example 1: Common Subsequence & LCS Definition

- A common subsequence of two strings is a subsequence that appears in both strings.
- A longest common subsequence (LCS) is a common subsequence of maximal length.
- **Example 1:**

S_1 = AAACCGTGAGTTATTCGTTCTAGAA

S_2 = CACCCCTAAGGTACCTTTGGTTC

LCS = ACCTAGTACTTTG

Example 2: Common Subsequences

- **Example 2: Two given sequences are:**

$X = \langle A, B, C, B, D, A, B \rangle$

$Y = \langle B, D, C, A, B, A \rangle$

- **Common subsequences of X and Y are:**

$\langle B, C, A \rangle$

length 3

$\langle B, D, A, B \rangle$

length 4 ✅ LCS

$\langle B, C, B, A \rangle$

length 4 ✅ LCS

- The sequences ⟨B, C, B, A⟩ and ⟨B, D, A, B⟩ of length 4 are LCS of X and Y,
- since X and Y have no common subsequence of length 5 or greater.

Formal Problem Definition

- In the longest-common-subsequence problem, the input is two sequences:

$$X = x_1, x_2, \ldots, x_m \quad \text{and} \quad Y = y_1, y_2, \ldots, y_n$$

- The goal is to find a maximum-length common subsequence of X and Y.
- This section shows how to efficiently solve the LCS problem using dynamic programming.

Overlapping Subproblems

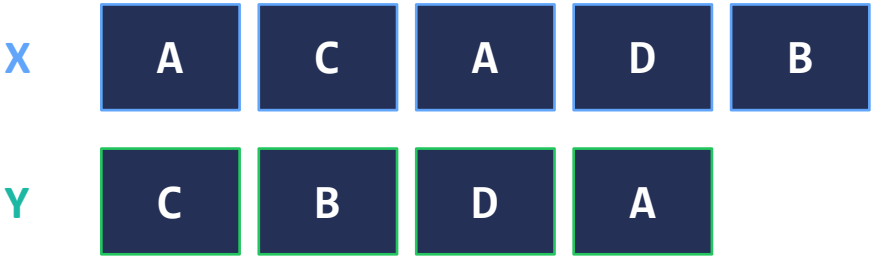
LCS(X[1..i], Y[1..j]) depends on smaller LCS subproblems — the same subproblems repeat many times.

Optimal Substructure

An optimal solution to LCS contains optimal solutions to subproblems — perfect for dynamic programming.

Walkthrough: Two Sequences

- Let us take two sequences:
- Our goal is to find the LCS using bottom-up dynamic programming.



Goal: Find LCS using bottom-up DP

Step 1: Initialize the Matrix

- **Step 1: Create a matrix of size (n+1)×(m+1) and fill the first column and row with 0.**
- X = ⟨A, C, A, D, B⟩ → m = 5 rows
- Y = ⟨C, B, D, A⟩ → n = 4 columns
- All border cells initialized to 0 (base case).

y_j

x_i		C	B	D	A
	0	0	0	0	0
A	0				
C	0				
A	0				
D	0				
B	0				

Fill the values →

Step 2: Fill Each Cell

- **Step 2: Compare x_i = 1,2,3,4,5 with each column y_1, y_2, y_3, y_4.**
- Match found → fill the current cell by adding 1 to the value of its left top diagonal cell. Arrow points to diagonal cell.
- No match → fill the current cell by taking max of previous row (top cell) and previous column (left cell), and point an arrow to the cell with maximum value. If they are equal, then point to any of them (we shall point to previous row (top cell).
- If top = left, point to top cell (↑).
- Note: always look at top and left *of the current cell*

y_j

		C	B	D	A
	0	0	0	0	0
A	0				
C	0				
A	0				
D	0				
B	0				

x_i

Fill the values →

Row 1: Compare x_1=A with y_1=C

- **Compare x_1=A (row 1) with each column y_1, y_2, y_3, y_4.**
- x_1=A and y_1=C do not match.
- → take max(top=0, left=0) = 0
- Values are equal, so arrow points to top cell (↑).
- A pointer is used to track the direction.

y_j

x_i		C	B	D	A
	0	0	0	0	0
A	0	0			
C	0				
A	0				
D	0				
B	0				

Fill the values →

Row 1: Compare x_1=A with y_2=B

- **Compare x_1=A (row 1) with y_2=B.**
- x_1=A and y_2=B do not match.
- → take max(top=0, left=0) = 0
- Values equal → arrow points to top cell (↑).

y_j

		C	B	D	A
	0	0	**0**	0	0
A	0	0	0		
C	0				
A	0				
D	0				
B	0				

x_i

Fill the values →

Row 1: Compare x_1=A with y_3=D

- **Compare x_1=A (row 1) with y_3=D.**
- x_1=A and y_3=D do not match.
- → take max(top=0, left=0) = 0
- Values equal → arrow points to top cell (↑).

y_j

x_i		C	B	D	A
	0	0	**0**	**0**	0
A	0	0	0	0	
C	0				
A	0				
D	0				
B	0				

Fill the values →

Row 1: Compare x_1=A with y_4=A ✅ Match!

- **Compare x_1=A (row 1) with y_4=A.**
- **x_1=A and y_4=A → MATCH! ✅**
- → diagonal cell value (0) + 1 = 1
- Arrow points diagonally (↖) to the top-left cell.

y_j

		C	B	D	A
	0	0	0	0	0
A	0	0	0	0	1
C	0				
A	0				
D	0				
B	0				

x_i

Fill the values →

Row 2: Compare x_2=C with all columns

- **Compare x_2=C (row 2) with y_1, y_2, y_3, y_4.**
- x_2=C and y_1=C → MATCH! dp[2][1] = dp[1][0]+1 = 1 ↖
- x_2=C and y_2=B → no match. max(top=1, left=1) = 1 ↑
- x_2=C and y_3=D → no match. max(top=0, left=1) = 1 ←
- x_2=C and y_4=A → no match. max(top=1, left=1) = 1 ↑

y_j

		C	B	D	A
	0	0	0	0	0
A	0	0	0	0	1
C	0	1	1	1	1
A	0				
D	0				
B	0				

x_i

Fill the values →

Row 3: Compare x_3=A with all columns

- **Compare x_3=A (row 3) with y_1,y_2,y_3,y_4.**
- x_3=A and y_1=C → no match. max(top=1, left=0) = 1 ↑
- x_3=A and y_2=B → no match. max(top=1, left=1) = 1 ↑
- x_3=A and y_3=D → no match. max(top=1, left=1) = 1 ↑
- x_3=A and y_4=A → MATCH! dp[3][4] = dp[2][3]+1 = 2 ↖

y_j

		C	B	D	A
	0	0	0	0	0
A	0	0	0	0	1
C	0	1	1	1	1
A	0	1	1	1	2
D	0				
B	0				

x_i

Fill the values →

Row 4: Compare x_4=D with all columns

- **Compare x_4=D (row 4) with y_1,y_2,y_3,y_4.**
- x_4=D and y_1=C → no match. max(top=1, left=0) = 1 ↑
- x_4=D and y_2=B → no match. max(top=1, left=1) = 1 ↑
- x_4=D and y_3=D → MATCH! dp[4][3] = dp[3][2]+1 = 2 ↖
- x_4=D and y_4=A → no match. max(top=2, left=2) = 2 ↑

y_j

		C	B	D	A
	0	0	0	0	0
A	0	0	0	0	1
C	0	1	1	1	1
A	0	1	1	1	2
D	0	1	1	2	2
B	0				

x_i

Fill the values →

Row 5: Compare x_5=B with all columns

- **Compare x_5=B (row 5) with y_1,y_2,y_3,y_4.**
- x_5=B and y_1=C → no match. max(top=1, left=0) = 1 ↑
- x_5=B and y_2=B → MATCH! dp[5][2] = dp[4][1]+1 = 2 ↖
- x_5=B and y_3=D → no match. max(top=2, left=2) = 2 ↑
- x_5=B and y_4=A → no match. max(top=2, left=2) = 2 ↑

y_j

		C	B	D	A
	0	0	0	0	0
A	0	0	0	0	1
C	0	1	1	1	1
A	0	1	1	1	2
D	0	1	1	2	2
B	0	1	2	2	2

x_i

Fill the values →

Step 3: Read the LCS Length

- **Step 3: The value in the last row and last column is the length of the LCS.**
- dp[5][4] = 2
- **→ The LCS has length 2.**
- The bottom-right cell always holds the final answer.

y_j

x_i		C	B	D	A
	0	0	0	0	0
A	0	0	0	0	1
C	0	1	1	1	1
A	0	1	1	1	2
D	0	1	1	2	2
B	0	1	2	2	2

Fill the values →

Step 4: Traceback to Find LCS

- **Step 4: Start from the last element dp[5][4] and follow the arrow directions.**
- Follow arrows back to the top-left corner.
- Collect characters at diagonal (↖) arrows — those form the LCS.

y_j

x_i		C	B	D	A
	0	0	0	0	0
A	0	0	0	0	1
C	0	1	1	1	1
A	0	1	1	1	2
D	0	1	1	2	2
B	0	1	2	2	2

Fill the values →

Traceback Result: LCS = ⟨C, A⟩

- The elements corresponding to the "↖" symbol form the LCS — select cells with diagonal arrow.
- **Here C and A have diagonal arrows (↖).**
- The longest common subsequence for:
- X = ⟨A, C, A, D, B⟩ and Y = ⟨C, B, D, A⟩
- **is ⟨C, A⟩ of length 2**

y_j

x_i		C	B	D	A
	0	0	0	0	0
A	0	0	0	0	1
C	0	1	1	1	1
A	0	1	1	1	2
D	0	1	1	2	2
B	0	1	2	2	2

Fill the values →

Answer: LCS = ⟨C, A⟩, Length = 2

LCS-LENGTH(X, Y, m, n) — Pseudocode

```
LCS-LENGTH(X, Y, m, n)
1   let b[1:m, 1:n] and c[0:m, 0:n] be new tables
2   for i = 1 to m
3       c[i, 0] = 0
4   for j = 0 to n
5       c[0, j] = 0
6   for i = 1 to m          // compute table entries in row-major
order
7       for j = 1 to n
8           if x_i == y_j
9               c[i, j] = c[i-1, j-1] + 1
10              b[i, j] = "↖"
11          elseif c[i-1, j] ≥ c[i, j-1]
12              c[i, j] = c[i-1, j]
13              b[i, j] = "↑"
14          else  c[i, j] = c[i, j-1]
15              b[i, j] = "←"
16  return c and b
```

Initialization

Create c[0..m,0..n] for lengths
and b[1..m,1..n] for directions.
Set all border cells to 0 (base case).

Characters Match (line 9-10)

c[i,j] = c[i-1,j-1]+1
Arrow points diagonally — character is part of LCS.

Top ≥ Left (line 12-13)

c[i,j] = c[i-1,j]
Arrow points upward.

Left > Top (line 14-15)

c[i,j] = c[i,j-1]
Arrow points left.

Initialization & Filling the Table

Initialization

- Inputs are sequences X and Y of lengths m and n.
- Create tables c[0..m, 0..n] and b[1..m, 1..n].
- Table c stores LCS lengths; table b stores arrow directions.

Base Case Setup

- Set the first column c[i, 0] and first row c[0, j] to 0.
- This represents LCS of empty strings = 0.

Filling the Table (Match case)

- If characters match (x_i == y_j):
- → set c[i,j] = c[i-1, j-1] + 1
- → point b[i,j] diagonally (↖)

No-match Case & Return Result

Filling the Table (No-Match case)

- If characters do not match, compare values from c[i-1, j] (top) and c[i, j-1] (left):
- If top ≥ left: c[i,j] = c[i-1,j] and b[i,j] = "↑"
- Otherwise: c[i,j] = c[i,j-1] and b[i,j] = "←"

Return Result

- Finally, return the tables c and b.
- c contains the LCS lengths for all subproblems.
- b contains the arrow directions needed for traceback.
- Time complexity: Θ(mn) — each cell takes Θ(1) to compute.

PRINT-LCS(b, X, i, j) — Pseudocode

```
PRINT-LCS(b, X, i, j)
1   if i == 0 or j == 0
2       return              // the LCS has length 0
3   if b[i, j] == "↖"
4       PRINT-LCS(b, X, i-1, j-1)
5       print x_i           // same as y_j
6   elseif b[i, j] == "↑"
7       PRINT-LCS(b, X, i-1, j)
8   else  PRINT-LCS(b, X, i, j-1)
```

Base Case

If i==0 or j==0, stop — reached the beginning of a sequence. LCS is empty.

↖ Diagonal

Characters match! Recurse on (i-1, j-1), then print x_i. This character is in the LCS.

↑ Up

x_i is not part of LCS. Recurse on row above (i-1, j).

← Left

y_j is not part of LCS. Recurse on column left (i, j-1).

Base Case & Diagonal / Up Arrows

Base Case Setup

- If either index i or j is zero, the function stops and returns.
- We've reached the end of one sequence — LCS contribution is empty here.

Handle Diagonal Arrow (↖)

- If b[i,j] == "↖", then x_i and y_j are part of the LCS.
- Recursively call PRINT-LCS(b, X, i-1, j-1) to handle earlier characters.
- **Then print x_i (which equals y_j).**

Handle Up Arrow (↑)

- If b[i,j] == "↑", then x_i is NOT part of the LCS.
- Recursively call PRINT-LCS(b, X, i-1, j) to move upward in the table.

Left Arrow & Summary

Handle Left Arrow (←)

- If b[i,j] == "←", then y_j is NOT part of the LCS.
- Recursively call PRINT-LCS(b, X, i, j-1) to move left in the table.

Traceback Flow Summary

Start at b[m,n]

b[i,j] == ↖?

Print x_i + recurse i-1,j-1

b[i,j] == ↑?

Recurse i-1, j

Else: Recurse i, j-1

Running Time & Worked Example Preview

- The figure in the next slide shows the tables produced by LCS-LENGTH on:
- **X = ⟨A, B, C, B, D, A, B⟩ and Y = ⟨B, D, C, A, B, A⟩**
- The running time of the procedure is Θ(mn), since each table entry takes Θ(1) time to compute.

Θ(mn)

Time: LCS-LENGTH

m=|X|, n=|Y|
Each of mn cells: O(1)

O(mn)

Space: Tables c & b

c[0..m,0..n] for lengths
b[1..m,1..n] for directions

O(m+n)

Time: PRINT-LCS

Each call decreases
i+j by at least 1

Floyd-Warshall Algorithm

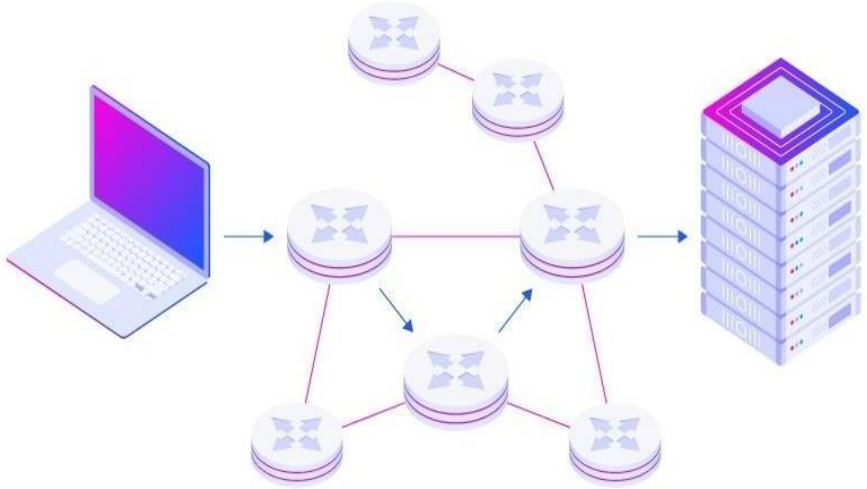
All-Pairs Shortest Path | Dynamic Programming

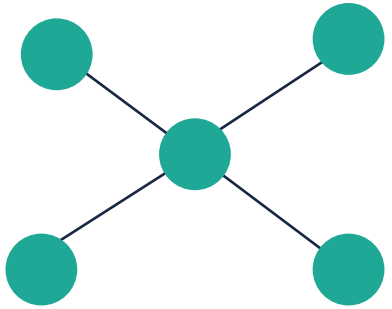
Type	Complexity	Graph
Dynamic Programming	**$O(V^3)$**	**Directed & Undirected**

- Finds shortest paths between ALL pairs of vertices
- Works with both directed and undirected weighted graphs
- Also known as Floyd's / Roy-Warshall / WFI algorithm

Application — Network Routing

- The Floyd-Warshall algorithm is used in computer networks to determine the shortest paths between all pairs of nodes.
- This is essential for efficient data transmission and routing packets through the network.



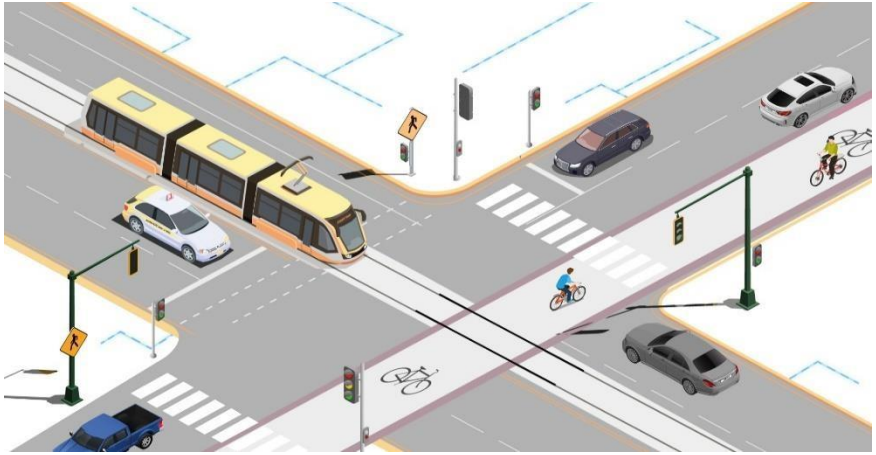


Application — Transportation & Logistics

- The Floyd-Warshall algorithm finds shortest paths between all location pairs in transportation systems, such as road networks or airline routes.
- This helps optimize routes for vehicles or flights, minimizing travel time and reducing fuel consumption.

Application — Traffic Management

- Urban planners and traffic engineers use Floyd-Warshall to analyze traffic flow and optimize signal timings at intersections.
- By finding shortest paths between city points, they develop strategies to alleviate congestion and improve overall traffic efficiency.



Application — Robotics

- Path planning algorithms in robotics utilize Floyd-Warshall to find shortest paths between various points in a robot's environment.

- This is crucial for autonomous robots navigating complex terrains or environments with obstacles.

Floyd-Warshall — Advantages

Handles Negative Weights

Works with weighted graphs containing both positive and negative edge weights.

Single Execution

One run of the algorithm is sufficient to find shortest paths between ALL pairs of vertices.

Path Reconstruction

Easy to modify the algorithm to reconstruct actual paths, not just distances.

Floyd-Warshall — Disadvantages

No Negative Cycles

The algorithm cannot find shortest paths when the graph contains negative cycles; the result becomes undefined.

No Path Details by Default

The basic algorithm only returns distances — it does not return the actual path sequences without modification.

Cubic Time Complexity

$O(V^3)$ time and $O(V^2)$ space complexity makes it impractical for very large graphs with many vertices.

Floyd-Warshall Algorithm

All-pairs shortest path using Dynamic Programming

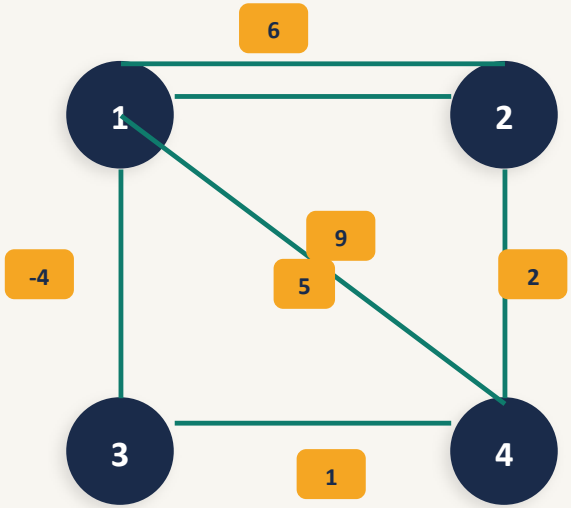
What it does

Computes the shortest path between ALL pairs of vertices in a weighted directed graph. Can handle negative edge weights (but not negative cycles).

Why DP?

Satisfies both DP conditions: optimal substructure (shortest sub-paths are optimal) and overlapping subproblems (intermediate vertex computations repeat).

Example Graph (4 vertices)



Algorithm Steps

1. **Step 1**
   Create distance matrix A^0 from the graph (direct edges, 0 on diagonal, ∞ otherwise)
2. **Step 2**
   For each intermediate vertex k (1→n): derive A^k from A^{k-1} by checking if path via k is shorter
3. **Step 3**
   Repeat Step 2 for every vertex k = 1 to n
4. **Step 4**
   Final matrix A^n contains all-pairs shortest paths

Step 1 — Create Distance Matrix A^0

Initialise with direct edge weights

Distance to itself (i = j)

$A^0[i,j] = 0$

Direct edge exists (i → j)

$A^0[i,j] = \text{weight of edge } w_{ij}$

No direct edge

$A^0[i,j] = \infty$

Recall the graph edges:

- 1→2: weight 9 ($A^0[1,2]=9$)
- 1→3: weight -4 ($A^0[1,3]=-4$)
- 3→2: weight 5 ($A^0[3,2]=5$)
- 2→1: weight 6 ($A^0[2,1]=6$)
- 2→4: weight 2 ($A^0[2,4]=2$)
- 4→3: weight 1 ($A^0[4,3]=1$)

Resulting A^0:

	1	2	3	4
1	0	9	-4	∞
2	6	0	∞	2
3	∞	5	0	∞
4	∞	∞	1	0

Diagonal = 0 | Edge weights from graph | ∞ = no direct path

Key Rule: if $A^0[i,j] > A^0[i,k] + A^0[k,j]$ → shorter path exists via intermediate vertex k → update!

Step 2 — Derive A^1 from A^0

Intermediate vertex k = 1: check if paths via vertex 1 are shorter

A^0 (previous)

	1	2	3	4
1	0	9	-4	∞
2	6	0	∞	2
3	∞	5	0	∞
4	∞	∞	1	0

k=1

A^1 (updated — via vertex 1)

	1	2	3	4
1	0	9	-4	∞
2	6	0	2	2
3	∞	5	0	∞
4	∞	∞	1	0

Key Check: Is there a shorter path from 2 → 3 via vertex 1?

- **Check $A^0[2,3] > A^0[2,1] + A^0[1,3]$** $\quad \infty > 6 + (-4) = 2$

  ✅ $\infty > 2$ → Update! $A^1[2,3] = 2$ (shorter path: 2→1→3)

- **Check $A^0[2,4] > A^0[2,1] + A^0[1,4]$** $\quad 2 > 6 + \infty = \infty$

  ❌ 2 is NOT $> \infty$ → Keep original $A^1[2,4] = 2$

- **Check $A^0[4,2] > A^0[4,1] + A^0[1,2]$** $\quad \infty > \infty + 9 = \infty$

  ❌ ∞ is NOT $> \infty$ → Keep original $A^1[4,2] = \infty$

Steps 3 & 4 — $A^2 \rightarrow A^3 \rightarrow A^4$ (Final Matrix)

Continue for k=2, 3, 4 until all intermediate vertices exhausted

A^2 (k=2)

	1	2	3	4
1	0	6	-4	∞
2	6	0	2	2
3	-4	2	0	∞
4	∞	2	7	0

A^3 (k=3)

	1	2	3	4
1	0	6	-4	12
2	1	0	2	6
3	-4	2	0	1
4	3	2	7	0

A^4 (Final)

	1	2	3	4
1	0	6	-4	12
2	1	0	2	6
3	-4	2	0	1
4	3	2	7	0

🏆 **Final Answer — A^4 gives the shortest path between every pair of vertices:**

e.g. $A^4[1,2] = 6$ (1→2 directly) | $A^4[2,3] = 2$ (2→1→3) | $A^4[4,1] = 3$ (4→3→1 via vertex 3)

Floyd - Warshall — Pseudocode & Java Implementation

Three nested loops — elegant and concise

Pseudocode

```
function FloydWarshall(graph):
  dist = n×n array
  // Step 1: Initialise A⁰
  for i from 1 to n:
    for j from 1 to n:
      if i == j:  dist[i][j] = 0
      else if edge(i,j) exists:
                  dist[i][j] = weight(i,j)
      else:       dist[i][j] = infinity
  // Step 2-4: Relax via each k
  for k from 1 to n:
    for i from 1 to n:
      for j from 1 to n:
        if dist[i][j] > dist[i][k]+dist[k][j]:
           dist[i][j] = dist[i][k]+dist[k][j]
  return dist
```

Java Implementation

```
public class FloydWarshall {
  final static int INF = 99999;
  public static void main(String[] args) {
    int[][] graph = {
      {0, 9, -4, INF},
      {6, 0, INF, 2},
      {INF, 5, 0, INF},
      {INF, INF, 1, 0}};
    int n = graph.length;
    int[][] dist = new int[n][n];
    for (int i=0; i<n; i++)
      for (int j=0; j<n; j++)
        dist[i][j] = (i==j)?0:(graph[i][j]!=0)?
                     graph[i][j]:INF;
    for (int k=0; k<n; k++)
      for (int i=0; i<n; i++)
        for (int j=0; j<n; j++)
          if (dist[i][j]>dist[i][k]+dist[k][j])
            dist[i][j]=dist[i][k]+dist[k][j];
  }
}
```

Complexity Analysis

Floyd-Warshall vs other shortest-path algorithms

⏱ Time Complexity — $O(n^3)$

Outer loop (k)	Middle loop (i)	Inner loop (j)
Iterates over n possible intermediate vertices	Iterates over n source vertices	Iterates over n destination vertices

```
Total iterations: n × n × n = n³
```

Time Complexity: $O(n^3)$

💾 Space Complexity — $O(n^2)$

The algorithm stores a single n×n distance matrix. No additional data structures are needed beyond this matrix.

Space Complexity: $O(n^2)$

📊 Comparison with Other Shortest-Path Algorithms

Algorithm	Scope	Negative Weights?	Time	Space
Dijkstra's	Single-source	❌ No	O((V+E)logV)	O(V)
Bellman-Ford	Single-source	✅ Yes	O(VE)	O(V)
Floyd-Warshall	All-pairs	✅ Yes	$O(V^3)$	$O(V^2)$

DP Problem-Solving Framework

5-Step method — from problem statement to AC

1 Define the State

What does dp[i] (or dp[i][j]) represent?
Write it as a comment before coding.

2 Define the Choices

What decisions are made at each step?
What options do you iterate over?

3 Write the Recurrence

Express dp[state] in terms of smaller subproblems.
This is the transition equation.

4 Set Base Cases

Initialise boundary values carefully.
Handle edge cases (n=0, empty string, etc.).

5 Optimise Space (if needed)

Can you reduce from $O(n^2) \rightarrow O(n) \rightarrow O(1)$?
Use rolling array technique.

Universal DP Template (Java)

```
// 1. State definition:  dp[i] = ...
// 2. Initialise dp array + set base cases
// 3. Iterate over states
// 4. Iterate over choices  →  apply recurrence
//    dp[state] = optimal(dp[sub-state] + cost)
// 5. return dp[target]
```

Practice & Common Pitfalls

LeetCode recommendations · Mistakes to avoid

Recommended LeetCode Problems

#509	Fibonacci Number	Easy	Warm-up
#322	Coin Change	Medium	1D DP
#1143	Longest Common Subsequence	Medium	2D DP
#300	Longest Increasing Subsequence	Medium	Classic
#72	Edit Distance	Medium	String DP
#416	Partition Equal Subset Sum	Medium	0/1 Knapsack

Common Pitfalls & Mistakes

✗ Vague state definition

dp[i] meaning is unclear → wrong transition. Always write "dp[i] = ..." as a comment first!

✗ Missing base cases

Forgetting dp[0] initialisation leads to index errors or wrong answers.

✗ Greedy vs DP confusion

Greedy local optimum ≠ global optimum. When in doubt — use DP.

Start with brute force

Code the naive recursive solution first → add memo → refactor to bottom-up.

SUMMARY

What Did You Learn?

- ✅ DP = Optimal Substructure + Overlapping Subproblems
- ✅ Top-Down (Memoization) vs Bottom-Up (Tabulation)
- ✅ Classic Problems: LCS(with Java code)
- ✅ Floyd-Warshall: all-pairs shortest path, matrix derivation $A^0 \rightarrow A^n$
- ✅ Pseudocode + Java implementation of Floyd-Warshall
- ✅ Time $O(n^3)$ · Space $O(n^2)$ · Handles negative weights
- ✅ 5-Step Framework: State → Choices → Recurrence → Base → Optimise